

PHYS 4027 / 5027 • FALL 2026 • LECTURE 4

Root finding, Interpolation



Prof. Alex Chen

Department of Physics

Washington University in St. Louis

Today

- **Finish root finding.** Newton and secant methods.
- **Interpolation.** Estimating $f(x)$ between known values, up to Lagrange's formula.
- **Tools.** VS Code, and how to work with Claude Code in this course.

Part I · Root finding, continued

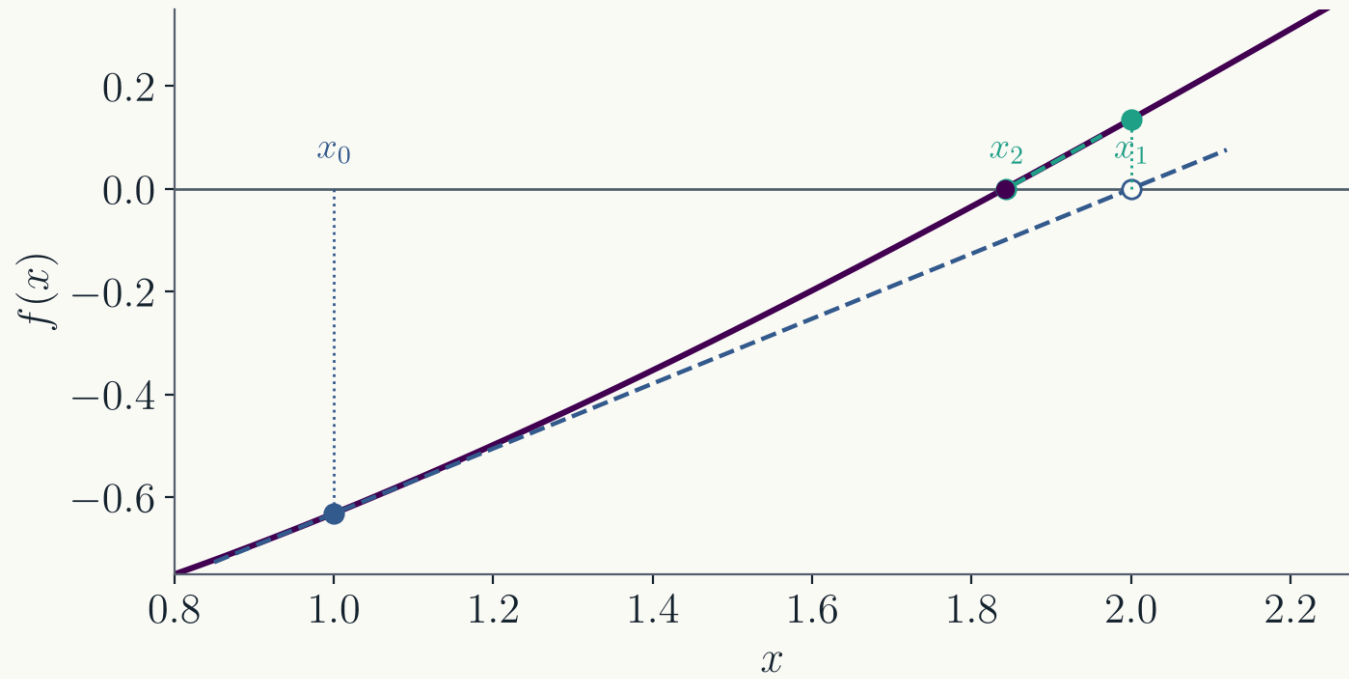


Newton · Secant · Writing the code

Where we were

- Solve $f(x) = 0$ by iterating: a sequence $x_0, x_1, x_2, \dots$ approaching the root x_* .
- **Fixed point.** Rewrite as $x = g(x)$ and iterate $x_{n+1} = g(x_n)$. Converges if $|g'(x_*)| < 1$; the error shrinks by that factor each step.
- **Bisection.** Keep an interval with a sign change, halve it. The error shrinks by $\frac{1}{2}$ each step, no matter what f is.
- Both are *linear*. Every iteration yields a fixed number of correct digits.

Newton's method



- Follow the tangent line at x_n down to the axis: $x_{n+1} = x_n - f(x_n)/f'(x_n)$.
- It is fixed-point iteration with $g(x) = x - f(x)/f'(x)$.

Newton's method



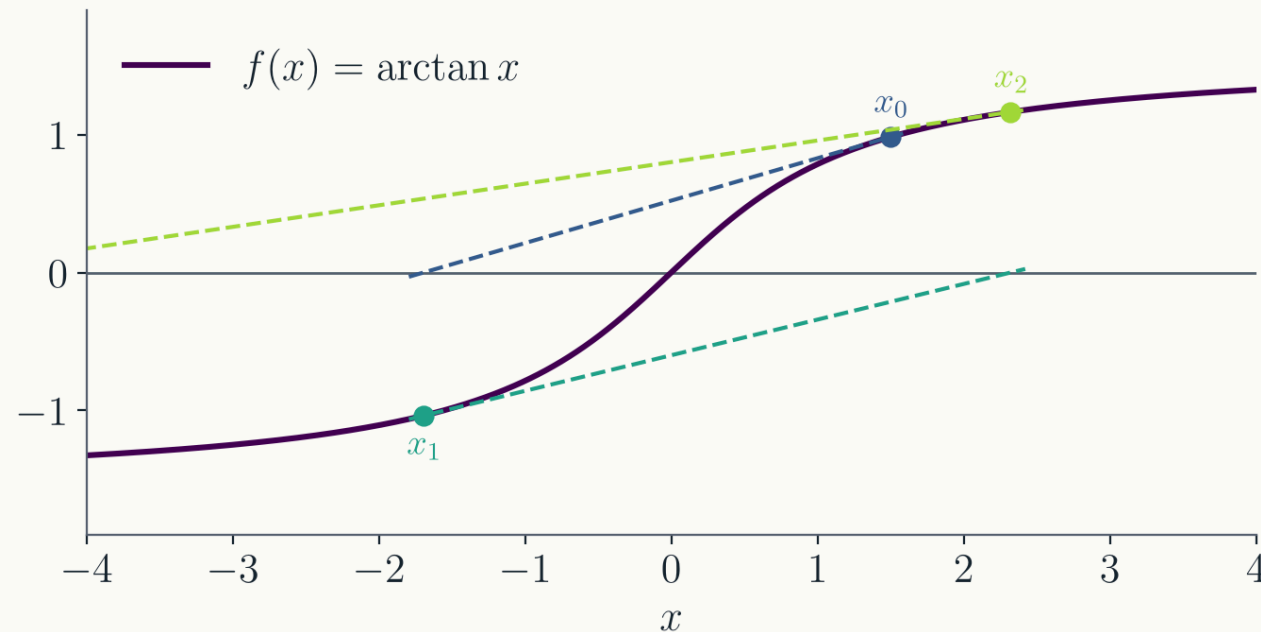
Taylor with remainder around x_n , evaluated at the root where $f(x_*) = 0$, gives:

$$x_{n+1} - x_* = \frac{f''(x_*)}{2f'(x_*)} (x_n - x_*)^2$$

- Expand $x_{n+1} = x_n - f(x_n)/f'(x_n)$ around x_* , keep terms up to $(x_n - x_*)^2$.
- The error is *squared* each step (quadratic convergence): k correct digits means $e_n \sim 10^{-k}$, and $e_n^2 \sim 10^{-2k}$ means $2k$ correct digits.
- In fixed-point language: $g'(x) = ff''/(f')^2$, so $g'(x_*) = 0$. The linear term is gone.

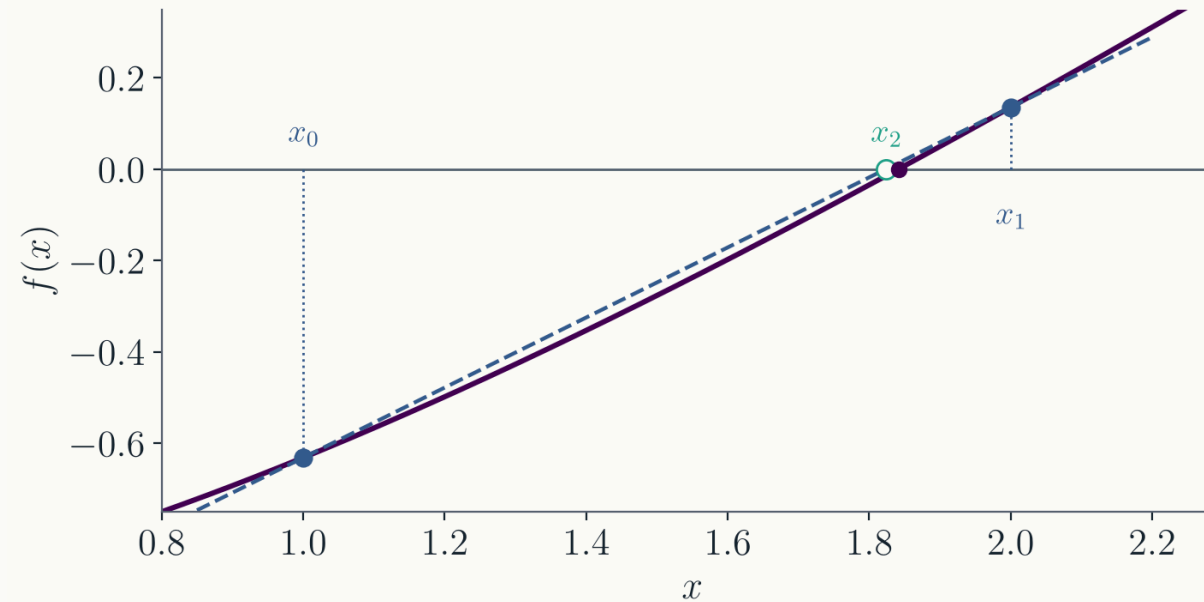
The price: it needs f' , and a starting point close enough to the root.

Drawbacks of Newton's method



- $f(x) = \arctan x$ from $x_0 = 1.5$: each tangent overshoots and lands farther away. From $x_0 = 1.3$ it converges.
- $f'(x_n) \approx 0$ sends x_{n+1} very far away. The iteration can also get stuck in a cycle.
- There is no bracket and no guarantee: fast near a root, unpredictable far from one.

Secant method



- No f' ? Use the slope of the line through the last two points instead of the tangent:

$$x_{n+1} = x_n - f(x_n) \frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}$$

- Needs two starting points, not a bracket, and only f .

Secant method



- Convergence order is the golden ratio, $\varphi = (1 + \sqrt{5})/2 \approx 1.62$: the error obeys $e_{n+1} \approx C e_n e_{n-1}$.
- Each step costs *one* new evaluation of f . Newton costs f and f' .
- Per function evaluation, secant is usually faster: two secant steps (two evaluations) give order $\varphi^2 \approx 2.6$; one Newton step (two evaluations) gives order 2.
- Shares Newton's failure modes, plus one more: if $f(x_n) = f(x_{n-1})$, the slope is 0/0.

Practical root finders combine a fast method with bisection as a fallback; Brent's method is the standard example.

Comparison

$|x_n - x_*|$ for $f(x) = x - 2 + e^{-x}$. Fixed point and Newton from $x_0 = 1$; bisection on $[1, 2]$; secant from $x_0 = 1, x_1 = 2$.

n	fixed point	bisection	secant	Newton
1	2.1×10^{-1}	9.1×10^{-2}	1.8×10^{-2}	1.6×10^{-1}
2	3.7×10^{-2}	3.4×10^{-2}	2.5×10^{-4}	2.1×10^{-3}
3	6.0×10^{-3}	2.9×10^{-2}	4.2×10^{-7}	4.1×10^{-7}
4	9.5×10^{-4}	2.3×10^{-3}	9.9×10^{-12}	1.6×10^{-14}
5	1.5×10^{-4}	1.3×10^{-2}	0	0

- Needs: a contracting g / a bracket / two starts / f' and a good start.
- Rate: linear (0.16) / linear (0.5, not monotone) / order 1.6 / order 2.

When to stop



- Three natural tests: the residual $|f(x_n)| < \varepsilon$; the step $|x_{n+1} - x_n| < \varepsilon$; the bracket $b - a < \varepsilon$.
- Absolute or relative? $|x_{n+1} - x_n| < \varepsilon |x_n|$ is the same test for a root at 10^{-3} and at 10^9 . Absolute tolerance only makes sense when you know the scale, or the root may be zero.
- **The floor.** Near $x_* = 1.84$, doubles are 2.2×10^{-16} apart (Lecture 1). Ask for $|x_{n+1} - x_n| < 10^{-20}$ and the loop never ends. The residual has a floor too: about $\epsilon |x_* f'(x_*)|$.

Do not ask for more digits than the format has.

Reporting failure

- Always set a maximum number of iterations; without one, a non-converging iteration loops forever.
- NaN propagates silently: `NaN < tol` is false, so a NaN iterate never triggers the stop test. Check with `std::isnan`.
- Return both the answer and whether the iteration converged:

```
std::tuple<bool, double> result = bisection(f, 1.0, 2.0, 1e-12);  
bool ok = std::get<0>(result);    // std::get<n>: the n-th tuple element  
double x = std::get<1>(result);  
if (!ok) { std::cout << "did not converge\n"; }
```

Passing a function to a function

Root-finding functions take f as an argument. This requires writing a function template:

```
template <typename F>                // F: anything callable as f(x)
double evaluate(const F& f, double x) {
    return f(x);
}

double square(double x) { return x * x; }
evaluate(square, 3.0);                // 9.0
```

The compiler generates a version of `evaluate` for every type `F` it is called with.

Lambda functions



A function can also be written with no name:

```
const double e = 0.3, M = 1.0;
auto kepler = [e, M](double E) { return E - e * std::sin(E) - M; };
evaluate(kepler, 1.0);

evaluate([](double x) { return x * x; }, 3.0);    // 9.0
```

- The brackets are the *capture list*: the outside variables the lambda may use.
- This is how a root finder gets an f with parameters in it: capture them.

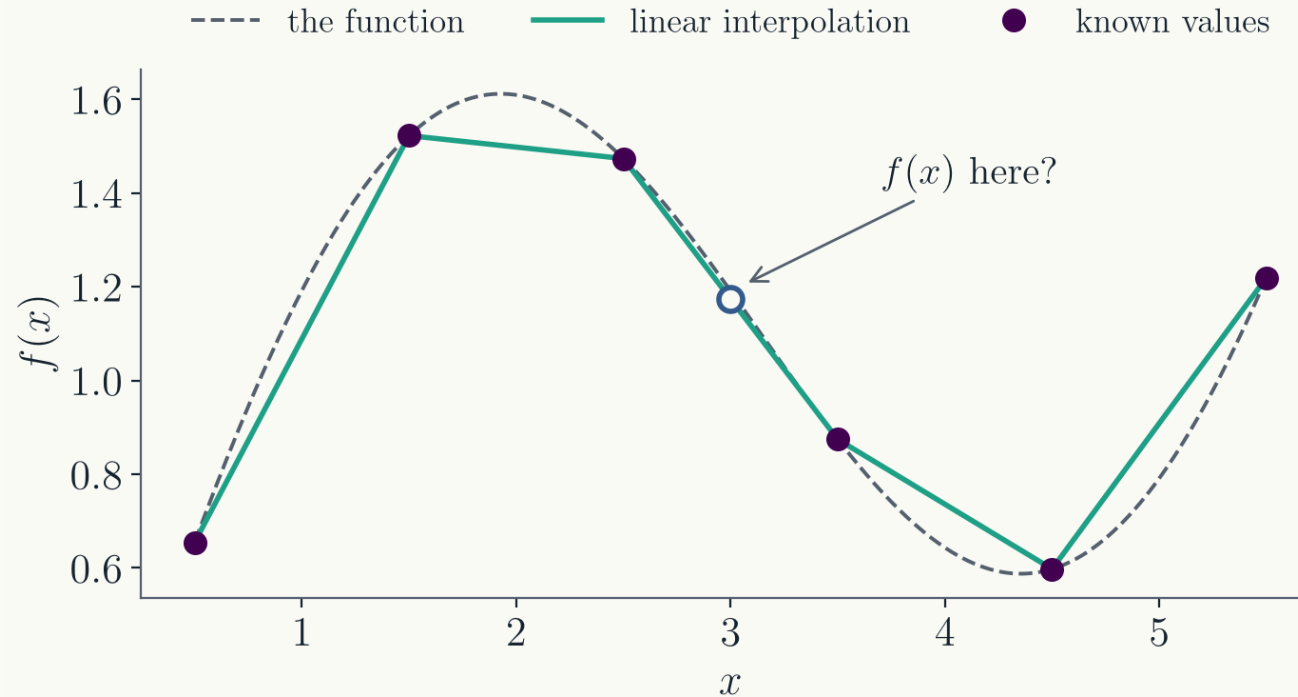
Part II · Interpolation



From two points to Lagrange's formula

Interpolation

So far $f(x)$ was a formula we could evaluate anywhere. Often you only have values at discrete points: tabulated material properties, output of a simulation on a grid, measured data.



Interpolation: estimate f between known values. Next lecture builds integration rules directly on top of it.

Linear interpolation



The simplest estimate: connect two known values with a straight line.

$$f(x) \approx f(x_0) + (x - x_0) \frac{f(x_1) - f(x_0)}{x_1 - x_0}$$

- Exact at x_0 and x_1 ; in between, it is the chord.
- What it misses is the curvature: the error grows with f'' and with the spacing $x_1 - x_0$.

More points



Rearrange the linear formula into a symmetric form:

$$f(x) \approx \frac{x - x_1}{x_0 - x_1} f(x_0) + \frac{x - x_0}{x_1 - x_0} f(x_1)$$

Each coefficient is 1 at its own node and 0 at the other. The same pattern works for three points:

$$f(x) \approx \frac{(x - x_1)(x - x_2)}{(x_0 - x_1)(x_0 - x_2)} f(x_0) + \frac{(x - x_0)(x - x_2)}{(x_1 - x_0)(x_1 - x_2)} f(x_1) + \frac{(x - x_0)(x - x_1)}{(x_2 - x_0)(x_2 - x_1)} f(x_2)$$

This is a parabola through three points: quadratic interpolation.

Lagrange's interpolation formula



For $N + 1$ points, the pattern gives

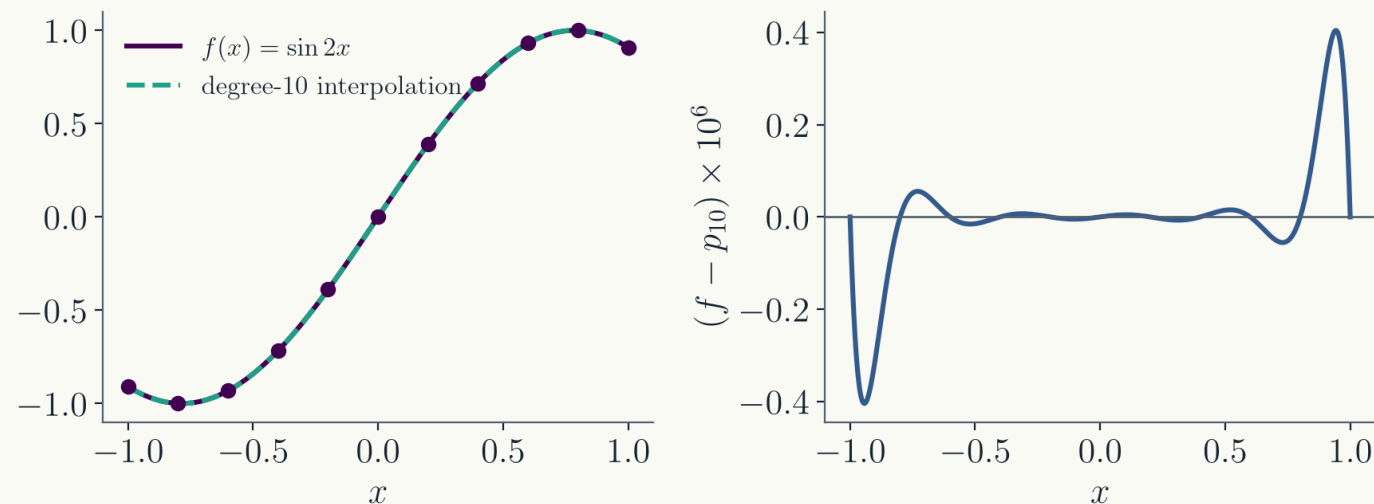
$$f(x) \approx \sum_{i=0}^N f(x_i) \prod_{\substack{j=0 \\ j \neq i}}^N \frac{x - x_j}{x_i - x_j}$$

- The x_i are called *nodes*. They need not be evenly spaced, only distinct.
- This is a polynomial of degree N through $N + 1$ points, and it is the *unique* one.

Unique means any method that fits a degree- N polynomial through these points produces this polynomial.

High-order interpolation

Take 11 equally spaced nodes on $[-1, 1]$ and fit one degree-10 polynomial through them. For $f(x) = \sin 2x$:

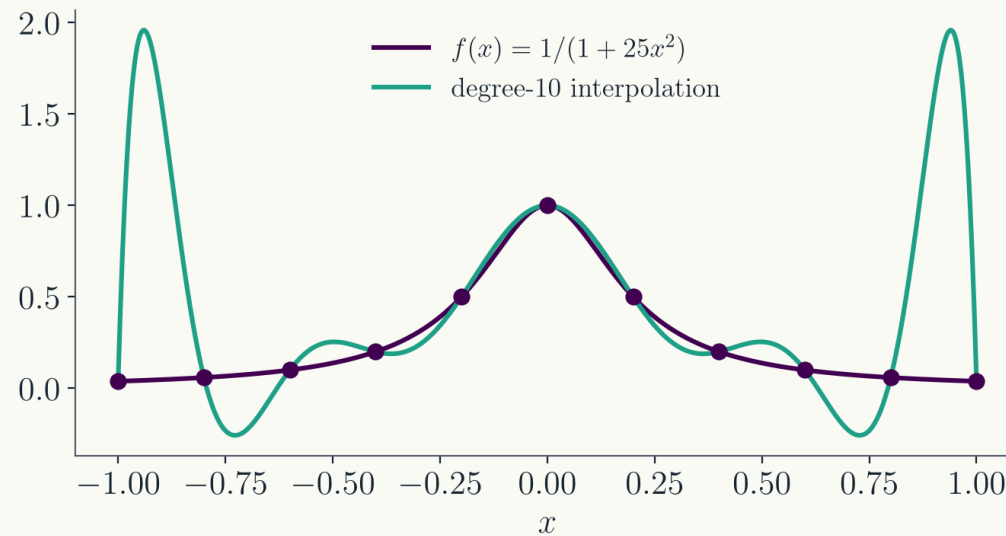


- The interpolant is indistinguishable from f : the largest error is 4×10^{-7} , from 11 samples.
- Note where the error is largest: near the ends of the interval.

Why not crank up the order for every function?

Runge's phenomenon

The same 11 nodes, now for $f(x) = 1/(1 + 25x^2)$:



- With equally spaced nodes, high-order interpolation can oscillate wildly near the ends of the interval.
- Adding more points makes it *worse*, not better.

Keep the order low and interpolate locally, a few points at a time. That is exactly what the integration rules next lecture do.

Part III · Tools



VS Code · Claude Code

VS Code habits

- Open your homework directory directly. This way version control directly knows about git.
- `Ctrl+P`: open a file by typing part of its name. `Ctrl+Shift+P`: every command by name.
- Red squiggles are compiler diagnostics before you compile. Hover one; it is usually the same message `g++` would print.
- The Source Control panel shows the diff against the last commit. Read it before you commit, especially when you did not type the change yourself.

macOS: `Cmd` in place of `Ctrl`.

Claude Code



- An AI agent that runs in the terminal, inside your repository. It does not just answer questions: it reads and edits files, compiles, runs, and uses git, showing each step for approval.
- Install and start (macOS, Linux, WSL):

```
curl -fsSL https://claude.ai/install.sh | bash  
cd hw1-yourname  
claude
```

- The first run asks you to log in.
- You can launch Claude Code in VS Code using an extension as well.

Directing an agent is a skill

- Be specific. "Implement the one-pass formula from the README, doubles everywhere, print both results" beats "do problem 2".
- Work in small steps and read what the agent has done. Do not commit code that you have not read.
- Make it show evidence: compile, run, and paste actual output. Ask questions about what it did and why it did it this way.
- It can be confidently wrong, even the latest models.

Where this goes next



- **Next lecture:** numerical integration, built from interpolation.
- References:
 - Numerical Recipes, 3rd ed., Chapter 3 (interpolation) and Chapter 9 (root finding).